

Assignment – 7

1. Create a new React app called InstaThemeDemo, then set up a ThemeContext with a Provider that manages a 'dark' or 'light' theme state.
2. Build a Navbar component and a deeply nested PostCard component (at least 3 levels deep) that both consume the theme value from ThemeContext and change their background color based on the current theme.
3. Add a ToggleThemeButton component that updates the theme in ThemeContext when clicked, so all components using the context update instantly.
4. Refactor your components so that NO theme prop is passed manually through intermediate components (no prop drilling) — only use ThemeContext to share the theme value.
5. Write a short explanation (3-5 lines) of how prop drilling would make sharing the theme state harder in your app, and how using ThemeContext solved this problem.

Hint: Focus on how many components would need to pass props versus just using context.

Answers:- 1. ThemeContext.jsx

```
import { createContext, useState } from "react";

export const ThemeContext = createContext();

const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState("light");

  const toggleTheme = () => {
    setTheme((prev) =>
      prev === "light" ? "dark" : "light"
    );
  };

  return (
    <ThemeContext.Provider
      value={{ theme, toggleTheme }}
    >
      {children}
    </ThemeContext.Provider>
  );
};

export default ThemeProvider;
```

2. Navbar.jsx

```

import { useContext } from "react";
import { ThemeContext } from "../context/ThemeContext";

const Navbar = () => {
  const { theme } = useContext(ThemeContext);

  return (
    <nav
      style={{
        padding: "20px",
        background:
          theme === "dark" ? "#222" : "#ddd",
        color:
          theme === "dark" ? "white" : "black",
      }}
    >
      Instagram Clone Navbar
    </nav>
  );
};

export default Navbar;

```

3. ToggleThemeButton.jsx

```

import { useContext } from "react";
import { ThemeContext } from "../context/ThemeContext";

const ToggleThemeButton = () => {
  const { toggleTheme } =
    useContext(ThemeContext);

  return (
    <button
      onClick={toggleTheme}
      style={{
        margin: "20px",
        padding: "10px",
      }}
    >
      Toggle Theme
    </button>
  );
};

export default ToggleThemeButton;

```

4. Feed.jsx

```
import PostList from "../PostList";

const Feed = () => {
  return (
    <div>
      <PostList />
    </div>
  );
};

export default Feed;
```

5. PostList.jsx

```
import PostCard from "../PostCard";

const PostList = () => {
  return (
    <div>
      <PostCard />
    </div>
  );
};

export default PostList;
```

6. PostCard.jsx

```
import { useContext } from "react";
import { ThemeContext } from "../context/ThemeContext";

const PostCard = () => {
  const { theme } = useContext(ThemeContext);

  return (
    <div
      style={{
        margin: "20px",
        padding: "20px",
        background:
          theme === "dark"
            ? "#333"
            : "#f4f4f4",
        color:
          theme === "dark"
```

```

      ? "white"
      : "black",
    }}
  >
  <h3>Post Card</h3>

  <p>
    This component is consuming ThemeContext
    without prop drilling.
  </p>
</div>
);
};

export default PostCard;

```

7. App.jsx

```

import ThemeProvider from "../context/ThemeContext";
import Navbar from "../components/Navbar";
import ToggleThemeButton from "../components/ToggleThemeButton";
import Feed from "../components/Feed";

function App() {
  return (
    <ThemeProvider>
      <Navbar />

      <ToggleThemeButton />

      <Feed />
    </ThemeProvider>
  );
}

export default App;

```

Props drilling : - Prop drilling occurs when data is passed through multiple intermediate components using props, even if those components do not need the data themselves. In this application, the theme would have to be passed from App → Feed → PostList → PostCard, making the code harder to maintain. Using ThemeContext allows any component to access the theme directly without manually

passing props through every level. This reduces complexity, improves readability, and makes state sharing much easier across deeply nested components.